PIM IV — ETAPA 6 · ARQUITETURA DE SOFTWARE

Manual Técnico — Backend C# e Aplicação Desktop

CodeRyse Academy / Sistema Acadêmico Integrado

.NET 10 | ASP.NET Core Web API | EF Core 10 | Electron (Desktop)

Documento: Manual_Tecnico_CSharp_Desktop.pdf

Data de elaboração: 23 de outubro de 2026

Método: auditoria direta do código-fonte do repositório (sem suposições) — toda afirmação técnica é referenciada a um arquivo e, quando aplicável, a uma linha

Repositório: Plataforma_Ensino_4-Semestre (branch `main`)

Sumário

1. Introdução e visão geral do sistema
2. Estrutura real da solução e organização das pastas
3. Ferramentas, tecnologias, frameworks e bibliotecas
4. Arquitetura implementada e responsabilidades das camadas
5. Funcionamento da aplicação desktop desde a inicialização
6. Fluxo de comunicação entre interface, API e banco de dados
7. Inventário de endpoints, parâmetros, respostas e regras de negócio
8. Classes, interfaces, métodos e dependências relevantes
9. Persistência, entidades, relacionamentos e migrations
10. Autenticação, autorização, segurança e configuração
11. Exemplos de fluxos reais extraídos do código
12. Limitações técnicas e funcionalidades parcialmente implementadas
13. Roteiro de apresentação acadêmica (10–15 min)
14. Perguntas prováveis da banca e respostas fundamentadas
15. Referências — arquivos e métodos utilizados
16. Glossário

Legenda usada no documento: **CONFIRMADO** *achado comprovado lendo o código real ·* **PARCIAL** *implementado parcialmente ou em transição ·* **PENDENTE** *não encontrado no código ou não verificável nesta auditoria.*

1. Introdução e visão geral do sistema

O **PlataformaEnsino** (nome comercial **CodeRyse Academy**) é uma plataforma EdTech completa, com backend em **.NET 10 / ASP.NET Core Web API** (projeto `Sistema Academico Integrado`, namespace raiz `PlataformaEnsino.API`) e frontend em React, construído com Vite e servido como arquivos estáticos pelo próprio backend. A partir desse mesmo backend, o sistema foi estendido para três superfícies de cliente adicionais: aplicativo **Mobile** (Expo/React Native), aplicativo **Desktop** (Electron) e um **Gateway** Node/Express opcional que atua como ponto de entrada único para Web/Mobile/Desktop. CONFIRMADO *Sistema Academico Integrado.csproj; Program.cs; desktop/main.js, desktop/package.json*

Este manual documenta especificamente a **parte C# (backend)** e a **aplicação Desktop**, por serem o escopo solicitado para a apresentação acadêmica da Etapa 6 do PIM IV (Arquitetura de Software). O frontend Web/React em si é tratado apenas na medida em que explica o funcionamento do Desktop (que o reutiliza integralmente) — não é objetivo deste documento detalhar componentes React.

> O projeto já foi auditado tecnicamente em sessões anteriores (ver arquivo `PIM 4ºSEM/especificacaoUML-Desktop-CSharp.md`, de 22/09/2026). Este manual **não reaproveita** aquele conteúdo sem verificação — o código evoluiu de forma relevante desde então (ex.: a tabela `ProfessoresCursos` e os campos `ModeloCurso`/encerramento de curso não existiam naquela data). Todas as afirmações aqui foram revalidadas contra o estado atual do repositório em 23/10/2026.

2. Estrutura real da solução e organização das pastas

A solução `Sistema Academico Integrado.sln` contém 2 projetos .NET: CONFIRMADO

- `Sistema Academico Integrado.csproj` — projeto principal (`Microsoft.NET.Sdk.Web` , `net10.0` , `OutputType=Exe`), que contém a API, as entidades e as migrations.
- `PlataformaEnsino.Tests/PlataformaEnsino.Tests.csproj` — projeto de testes (xUnit), explicitamente excluído da compilação do projeto principal (`Compile Remove="PlataformaEnsino.Tests\**\*.cs"`).

Pastas do projeto principal (contagem real de arquivos)

Pasta	Qtde.	Conteúdo
`Controllers/`	15	Camada HTTP — um controller por recurso de domínio.
`Services/`	31	Regras de negócio.
`Interfaces/`	30	Contratos de Services e Repositories (injeção de dependência).
`Repositories/`	5	`GenericRepository<T>` + 4 repositórios específicos (Conteúdo Didático, Matrícula, Módulo, Turma).
`Models/`	29	Entidades de domínio (EF Core).
`DTOs/`	65	Contratos de request/response — entidades nunca são expostas diretamente na API.
`Data/`	—	`PlataformaContext.cs` (DbContext), `DevelopmentDataSeeder.cs` , subpasta `Configurations/` com 7 arquivos de Fluent API.
`Migrations/`	36	Histórico completo de migrations EF Core, de 25/03/2026 até 22/10/2026.
`Common/`	9	Middleware de erro, middleware de log, extensions de `ClaimsPrincipal` , geração de código de registro, health check, validação de usuário.
`Storage/`	—	Diretório de upload local (apenas quando Azure Blob Storage não está configurado — ver seção 10).

Fora da solução .NET (irmãos no mesmo repositório)

Pasta	Stack	Papel
`frontend/`	React + Vite	SPA Web; build (`npm run build`) sai direto em `wwwroot/` , na raiz do backend.
`gateway/`	Node/Express	Reverse proxy puro na frente da API (repassa `/api/*` e `/uploads/*` , sem lógica própria).
`mobile/`	Expo / React Native	App mobile (Android), fala direto com o Gateway.
`desktop/`	Electron	Wrapper nativo da mesma SPA Web — ver seção 5.

3. Ferramentas, tecnologias, frameworks e bibliotecas

Lista extraída diretamente de `Sistema Academico Integrado.csproj`, sem suposição de versão. **CONFIRMADO**

Categoria	Pacote	Versão	Papel
Runtime / linguagem	`TargetFramework`	net10.0	.NET 10, C# com `Nullable` e `ImplicitUsings` habilitados.
	SDK	Microsoft.NET.Sdk.Web	Projeto Web API (não console puro, apesar de `OutputType=Exe`).
Persistência	Microsoft.EntityFrameworkCore.SqlServer	10.0.5	Provider EF Core → SQL Server.
	Microsoft.EntityFrameworkCore.Design / .Tools	10.0.5	Geração/aplicação de migrations via CLI.
	BCrypt.Net-Next	4.1.0	Hash de senha (adaptativo, com salt).
Autenticação	Microsoft.AspNetCore.Authentication.JwtBearer	10.0.5	Validação de token JWT (Bearer).
	(sem pacote externo)	—	Refresh token é implementação própria (hash SHA-256 persistido, sem biblioteca de terceiros).
Documentação de API	Microsoft.AspNetCore.OpenApi / Microsoft.OpenApi	10.0.5 / 2.7.6	Geração nativa de OpenAPI (sem Swashbuckle).
	Scalar.AspNetCore	2.4.1	UI de documentação interativa em `/scalar/v1` (substitui Swagger UI).
Armazenamento em nuvem	Azure.Storage.Blobs	12.30.1	Upload de arquivos em produção (disco efêmero do App Service).
	Azure.Extensions.AspNetCore.DataProtection.Blobs	1.5.4	Persiste chaves de Data Protection no Blob Storage (evita invalidação a cada reinício/escala).
Processamento de documento	ClosedXML	0.105.1	Exportação de relatório de desempenho em Excel (`.xlsx`).
	DocumentFormat.OpenXml	3.5.1	Suporte a formatos Office (dependência do ClosedXML).
	UglyToad.PdfPig	1.7.0-custom-5	Extração de texto de PDF (geração de questões por IA a partir de material didático).
E-mail	MailKit	4.18.1	Envio de e-mail de recuperação de senha.
Segurança (pin direto)	System.Security.Cryptography.Pkcs	10.0.13	Fixado explicitamente para evitar versão transitiva vulnerável a DoS (comentário no `.csproj` cita CVEs específicos).
Testes	xUnit	(via PlataformaEnsino.Tests.csproj)	22 arquivos de teste, cobrindo Services críticos (Auth, Matrícula, Avaliação, Certificado, Pagamento, IA, etc.).

Integrações externas (via configuração, não pacote SDK dedicado)

Geração de questões por IA: provedor trocável entre **Anthropic** (padrão) e **Google Gemini** via chave de configuração `IA:Provedor`, decidido uma única vez na inicialização (`Program.cs:86-94`) — não é hot-swap em runtime. Ambos chamados via `IHttpClientFactory` puro (sem SDK de terceiros), contra `api.anthropic.com` e `generativelanguage.googleapis.com` respectivamente. **CONFIRMADO** *Program.cs:86-107*

Ferramenta de modelagem UML

PENDENTE — não há nenhuma ferramenta de modelagem UML integrada ao projeto ou referenciada no código/configuração. A criação dos diagramas é tratada à parte, no Manual 2 deste pacote (*Manual_Orientacao_Diagramas_UML.pdf*), usando uma ferramenta externa (Astah, Visual Paradigm, StarUML, Enterprise Architect ou diagrams.net, à escolha do desenvolvedor).

4. Arquitetura implementada e responsabilidades das camadas

A arquitetura é em camadas, com **injeção de dependência baseada em interface** registrada centralmente em `Program.cs` (30 registros `AddScoped<IXxxService, XxxService>` ou `IXxxRepository, XxxRepository`, mais `IGenericRepository<T>` genérico). CONFIRMADO *Program.cs:58-141*

```
Controllers (HTTP)
   │  depende de → I*Service (interface)
   ▼
Services (regra de negócio)
   │  depende de → I*Repository (interface) OU PlataformaContext direto
   ▼
Repositories (acesso a dados)  ──OU──  PlataformaContext direto
   │  usa → DbSet<T> (EF Core)
   ▼
Models (entidades de domínio)

Pacotes transversais:
  DTOs         → contratos de entrada/saída, nunca expõem entidade direto
  Interfaces   → contratos de todos os Services/Repositories
  Common       → ApiExceptionMiddleware, RequestLoggingMiddleware, extensions
```

Padrão híbrido de acesso a dados (achado relevante, não estava no documento de referência anterior)

> **Repository não é usado para todas as entidades.** Apenas 4 entidades têm repositório dedicado com query customizada (`IConteudoDidaticoRepository`, `IMatriculaRepository`, `IModuloRepository`, `ITurmaRepository`). Para as demais, os Services injetam `PlataformaContext` diretamente — por exemplo, `CursoService` recebe `PlataformaContext` + `IGenericRepository<Curso>` + `IGenericRepository<Coordenador>` + `INotificacaoService`, usando o repositório genérico só para CRUD simples e o `DbContext` direto para qualquer consulta com `.Where` / `.Include` mais elaborada. CONFIRMADO *Program.cs:58-61,68; Services/CursoService.cs:10-27* Essa é uma decisão de design já documentada em auditorias anteriores do projeto como deliberada (repositório dedicado só quando há consulta reutilizável/complexa o bastante pra justificar a camada extra) — ver seção 12 para o detalhamento como limitação/decisão arquitetural.

Por que isso é modularização real

Cada camada depende apenas da **interface** da camada abaixo, nunca da implementação concreta — `CursosController` conhece `ICursoService`, não `CursoService`. Trocar a implementação (ex.: trocar o provider de IA, como já acontece entre `AvaliacaoIAService` / `GeminiAvaliacaoIAService`) não exige alterar o controller nem o service que o consome. Isso é Inversão de Dependência (o "D" do SOLID), resolvida via injeção de dependência nativa do ASP.NET Core.

5. Funcionamento da aplicação desktop desde a inicialização

O cliente Desktop é implementado em `desktop/` com **Electron 32** (dependência de desenvolvimento, empacotada via **electron-builder 25**). O pacote publicado contém **apenas** `main.js` (86 linhas) — nenhum código React nem C# é compilado para dentro do instalador; o app carrega, em tempo de execução, a mesma SPA que o backend serve para o navegador. CONFIRMADO *desktop/package.json:14-23; desktop/main.js*

Sequência de inicialização (arquivo lido integralmente)

1. `app.whenReady()` dispara dois passos: registra uma **Content-Security-Policy** restritiva via `session.defaultSession.webRequest.onHeadersReceived` (`default-src 'self'`, `script-src 'self'`, `connect-src 'self'`, `object-src 'none'`, `frame-ancestors 'none'`; `style-src` permite `'unsafe-inline'` porque a SPA React/framer-motion injeta estilo inline em runtime) e chama `criarJanelaPrincipal()`. *main.js:51-73*
2. `criarJanelaPrincipal()` instancia um `BrowserWindow` (1280×800, mínimo 960×600) com **endurecimento de segurança explícito**: `contextIsolation: true`, `nodeIntegration: false`, `sandbox: true` — o processo de renderização não tem acesso a nenhuma API do Node.js. *main.js:9-21*
3. Define um handler para `target="_blank"` (ex.: material didático do tipo "Link"): em vez de abrir uma nova janela Electron, abre no navegador padrão do sistema via `shell.openExternal`. *main.js:26-30*
4. Define um guard de navegação de topo (`will-navigate`): qualquer tentativa de navegar para uma origem diferente da URL carregada é bloqueada e redirecionada para o navegador externo — impede que a janela Electron vire um navegador genérico. *main.js:32-39*
5. `janela.loadURL(APP_URL)` carrega a URL alvo. *main.js:41*

Como `APP_URL` é resolvida

```
const APP_URL = process.env.DESKTOP_APP_URL || "https://app-coderyse-academy.azurewebsites.net";
```

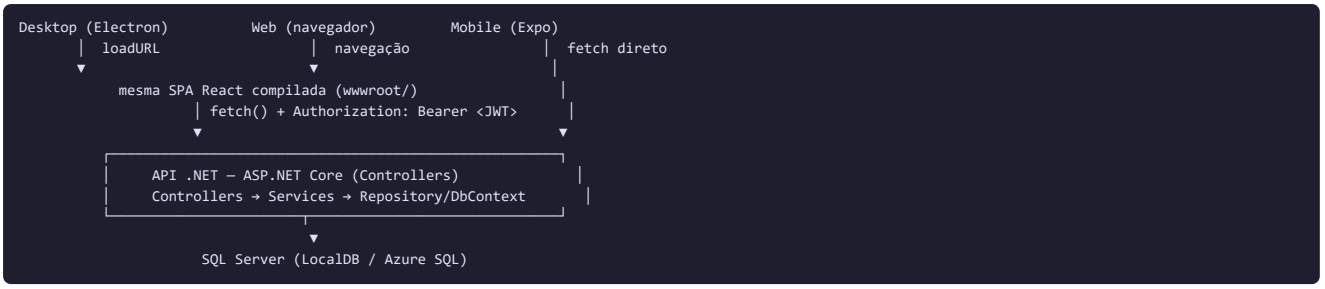
O padrão aponta para a **API em produção no Azure** — decisão corrigida nesta própria sessão de trabalho (commit `d9aef5f`, 23/10/2026): antes o padrão era `http://localhost:5000`, o que tornava qualquer instalador distribuído inutilizável (o usuário final não tem a API rodando na própria máquina). Para desenvolvimento local, a variável de ambiente `DESKTOP_APP_URL` deve ser definida manualmente apontando para `http://localhost:5000` antes de `npm start`. CONFIRMADO *desktop/main.js:4-11; desktop/README.md*

Empacotamento do instalador

O instalador nativo nasce de `electron-builder` via `npm run build` dentro de `desktop/`: gera `.exe` (NSIS) no Windows, `AppImage` no Linux, `.dmg` no macOS. CONFIRMADO *desktop/package.json:12,18-32* A SPA React é construída separadamente pelo **Vite** (`frontend/npm run build`), com saída direto em `wwwroot/` na raiz do backend (`vite.config.js`) — o Electron **não compila** nenhum código React; ele só consome o HTML/JS/CSS já publicado pelo servidor que carrega.

6. Fluxo de comunicação entre a interface, a API e o banco de dados

> **Cuidado explícito pedido na auditoria:** o processo principal do Electron (`main.js`) **não contém nenhuma chamada HTTP, nenhum uso de `fetch` / `XHR` / `axios`, nem qualquer manipulação de token JWT** — isso foi verificado lendo o arquivo por inteiro. "O Desktop usa JWT" só é verdadeiro no sentido indireto de que ele carrega a mesma SPA que já faz isso — a lógica de autenticação é 100% do código React compartilhado com a Web, não código específico do Electron.



Fluxo real de uma requisição autenticada, confirmado em `Program.cs` : `UseRequestLoggingMiddleware` → `UseStaticFiles` → `UseApiExceptionMiddleware` → `UseCors` → `UseAuthentication` → `UseAuthorization` → `UseRateLimiter` → roteamento para o `Controller` correspondente. **CONFIRMADO** *Program.cs:310-318*

Em desenvolvimento local, Web e Mobile normalmente passam por um **Gateway** Node/Express (`:4000`) antes da API (`:5000`) — reverse proxy puro, sem lógica de negócio, repassando `/api/*` e o header `Authorization` . O Desktop, ao carregar a build de produção, fala **diretamente** com a API (mesmo host que serve o HTML), sem passar pelo Gateway — não há evidência de uso do Gateway pelo Desktop no código lido.

7. Inventário de endpoints, parâmetros, respostas e regras de negócio

Auditoria exaustiva dos 15 controllers. Todas as rotas usam o prefixo `api/v1/[controller]` (convenção ASP.NET Core). CONFIRMADO

7.1 Tratamento de erro global

Exception	Status HTTP	Log
`KeyNotFoundException`	404 Not Found	Warning
`UnauthorizedAccessException`	401 Unauthorized	Warning
`ArgumentException`	400 Bad Request	Warning
`InvalidOperationException`	422 Unprocessable Entity	Warning
`Exception` (genérica)	500 Internal Server Error — mensagem fixa "Ocorreu um erro interno ao processar a solicitação."	Error

Payload padronizado: `{ erro, status, correlationId }`. Common/ApiExceptionMiddleware.cs:17-48; Common/RequestLoggingMiddleware.cs

7.2 Extensões de autorização usadas em todos os controllers

- `ObterUsuarioId()` — lê a claim `NameIdentifier`, com fallback para claim própria `"usuarioId"`.
- `PodeAcessarAluno(alunoId)` — `true` se o usuário é Admin/Coordenador, senão só se o id do token bate com o id solicitado (bloqueia aluno acessando dado de outro aluno).

Common/ClaimsPrincipalExtensions.cs:10-27

7.3 Rate limiting

Policy	Limite	Chave de partição	Aplicada em
`"auth"`	10 req/min	IP	AuthController: login, refresh, esqueci-senha, redefinir-senha, bootstrap-admin
`"ia-geracao"`	10 req/hora	usuarioId (claim), fallback IP	AvaliacoesController: gerar-questoes, gerar-questoes-de-conteudo

7.4 Endpoints públicos (ator Visitante)

Rota	Observação
`POST /Auth/login`, `/refresh`, `/logout`, `/esqueci-senha`, `/redefinir-senha`, `/bootstrap-admin`	`bootstrap-admin` é de uso único — bloqueia após a 1ª chamada.
`GET /Certificados/verificar/{codigo}`	Verificação pública de autenticidade de certificado.
`GET /Cursos` e `GET /Cursos/{id}`	`CursosController` não tem `[Authorize]` de classe — esses dois GETs são públicos por ausência de atributo, não por design explícito documentado em comentário.
`POST /Alunos/cadastro-completo`	Auto-cadastro público (cria conta + matrícula automática).

7.5 Tabela completa de endpoints por controller

AuthController (`api/v1/Auth`) — sem `[Authorize]` de classe, todos `[AllowAnonymous]`

Rota	Verbo	Request	Response	Descrição
/login	POST	LoginDto	AuthResponseDto / 401	Login, emite JWT + refresh token
/refresh	POST	RefreshTokenDto	AuthResponseDto / 401	Renova JWT via refresh token (rotação)
/logout	POST	RefreshTokenDto	200	Revoga refresh token
/esqueci-senha	POST	EsqueciSenhaDto	200 (msg genérica)	Dispara e-mail de recuperação
/redefinir-senha	POST	RedefinirSenhaDto	200	Redefine senha via token
/bootstrap-admin	POST	BootstrapAdminDto	AuthResponseDto	Cria o 1º Admin (uso único)

UsuariosController (`api/v1/Usuarios`) — classe `[Authorize]`

Rota	Verbo	Auth	Descrição
/	GET	Admin	Lista todos usuários
/{id}	GET	Admin	Usuário por id
/me	GET / PUT	autenticado	Perfil próprio
/me/senha	PUT	autenticado	Troca de senha própria
/{alunoId}/feedbacks	POST	Professor	Cria feedback acadêmico sobre o aluno
/{alunoId}/feedbacks	GET	autenticado	Lista feedbacks (escopo por quem chama)
/{alunoId}/feedbacks/{id}/lido	PUT	Aluno	Marca feedback como lido

AlunosController (`api/v1/Alunos`) — sem `[Authorize]` de classe

Rota	Verbo	Auth	Descrição

/	GET	Admin,Coordenador	Lista alunos (paginação opcional)
/	POST	Admin,Coordenador	Cria aluno (fluxo administrativo)
/cadastro-completo	POST	público	Auto-cadastro + matrícula automática

ProfessoresController / CoordenadoresController

CRUD completo (`GET` / `POST` / `PUT {id}` / `DELETE {id}`). `ProfessoresController` : classe `[Authorize(Roles="Admin,Coordenador")]` . `CoordenadoresController` : classe `[Authorize(Roles="Admin")]` — único recurso gerenciável só por Admin, sem Coordenador.

CursosController (`api/v1/Cursos`) — sem `[Authorize]` de classe

Rota	Verbo	Auth	Descrição
/	POST	Admin,Coordenador	Cria curso
/	GET	público	Lista todos os cursos
/meus	GET	Professor	Cursos do professor logado (via ProfessorCurso)
/{id}	GET	público	Curso por id
/{id}	PUT	Admin,Coordenador + autorização	Atualiza curso; Coordenador bloqueado de mudar Preço (403)
/{id}/coordenador	PUT	Admin	Atribui/remove coordenador
/{id}/professores/{professorId}	POST / DELETE	Admin,Coordenador + autorização	Vincula/desvincula professor ↔ curso (N:N)
/{id}/professores	GET	Admin,Coordenador	Lista professores vinculados
/{id}/concluir	POST	Professor	Marca sua parte concluída (curso Acompanhado)
/{id}/encerrar	POST	**Coordenador** (não Admin)	Encerra curso Acompanhado
/{id}/reabrir	POST	Coordenador	Reabre curso encerrado
/{id}/imagem	POST	Admin + autorização	Upload de capa (limite 6MB)
/taxa-adesao	GET	Admin,Coordenador	Métrica de adesão (escopo por role)
/desempenho, /{id}/desempenho	GET	Coordenador,Professor	Desempenho agregado

ModulosController (`api/v1/Modulos`) — classe `[Authorize]`

Rota	Verbo	Auth	Descrição
/	GET	autenticado	Lista com contagem de conteúdos/avaliações
/meus	GET	Professor	Módulos dos cursos do professor
/aluno/{alunoId}	GET	Aluno + PodeAcessarAluno	Módulos visíveis ao aluno
/{id}, /curso/{cursoId}	GET	autenticado	Módulo por id / por curso
/	POST	Admin,Coordenador + autorização	Cria módulo
/{id}	PUT / DELETE	Admin,Coordenador + autorização	Atualiza / exclui

TurmasController (`api/v1/Turmas`) — classe `[Authorize]`

Rota	Verbo	Auth	Descrição
/	POST	Admin,Coordenador + autorização	Cria turma
/{id}, /	GET	autenticado	Turma por id / lista geral
/minhas, /desempenho	GET	Professor	Turmas / desempenho do professor
/{id}/desempenho/exportar	GET	Professor	Exporta `.xlsx` (ClosedXML)
/{id}/professor	PUT	Admin,Coordenador + autorização	Atribui professor responsável
/{id}	PUT / DELETE	Admin,Coordenador + autorização	Renomeia / exclui

MatriculasController (`api/v1/Matriculas`) — classe `[Authorize]`

Rota	Verbo	Auth	Descrição
/	GET	Admin,Coordenador	Lista matrículas (Coordenador só dos próprios cursos)
/pendentes	GET	Admin,Coordenador	Matrículas pendentes
/aluno/{alunoId}, /{id}	GET	autenticado + PodeAcessarAluno	Matrículas de um aluno / por id
/	POST	autenticado + PodeAcessarAluno	Solicita matrícula (aprovação automática quando aplicável)
/{id}/aprovar, /aprovar-lote	PUT	Admin,Coordenador (+ autorização)	Aprova individual / em lote
/{id}/rejeitar	PUT	Admin,Coordenador + autorização	Rejeita
/{id}/cancelar, /{id}/reabrir	PUT	autenticado + PodeAcessarAluno	Aluno cancela/reabre a própria solicitação

ConteudosDidaticosController (`api/v1/ConteudosDidaticos`) — classe `[Authorize]`

Rota	Verbo	Auth	Descrição
/	GET	Professor	Conteúdos do professor
/aluno/{alunoId}	GET	Aluno + PodeAcessarAluno	Conteúdos visíveis ao aluno
/{id}	GET	Professor	Conteúdo por id (só do próprio professor)
/arquivo	POST	Professor	Upload de arquivo (limites por tipo: imagem 5MB, PDF 20MB, vídeo 100MB)
/	POST	Professor	Cria conteúdo
/{id}	PUT / DELETE	Professor	Atualiza / exclui

AvaliacoesController (`api/v1/Avaliacoes`) — classe `[Authorize]` , o maior controller

Rota	Verbo	Auth	Descrição
/, /{id}	GET	Professor	Avaliações/quizzes do professor / por id
/aluno/{alunoId}	GET	Aluno + PodeAcessarAluno	Avaliações visíveis ao aluno
/	POST	Professor	Cria avaliação/quiz
/{id}	PUT / DELETE	Professor	Atualiza / exclui
/{id}/questoes	GET (visão professor) / POST	Professor	Lista com gabarito / adiciona questão
/{id}/aluno/questoes	GET	Aluno	Questões pro aluno responder (sem gabarito)
/{id}/questoes/lote	POST	Professor	Confirma lote de questões (tudo-ou-nada, fluxo pós-IA)
/{id}/questoes/{qId}	PUT / DELETE	Professor	Atualiza / exclui questão
/{id}/questoes/ordem	PUT	Professor	Reordena questões
/{id}/aluno/respostas	POST	Aluno	**Envia respostas, cria tentativa** — único lugar que valida limite de tentativas
/{id}/aluno/tentativas/{tId}/revisao	GET	Aluno	**Consulta revisão de tentativa já concluída** — endpoint separado, sem checagem de limite de tentativas
/{id}/ia/gerar-questoes	POST	Professor, rate `ia-geracao`	Gera questões via IA a partir de upload (não persiste)
/{id}/ia/gerar-questoes-de-conteudo	POST	Professor, rate `ia-geracao`	Gera via IA a partir de Conteúdo Didático já cadastrado
/questoes-banco/{qbId}/anexos	POST / DELETE	Professor	Anexa/remove arquivo em questão do banco

PagamentosController / ProgressosController — ambos classe `[Authorize(Roles="Aluno")]`

Rota	Verbo	Descrição
/Pagamentos/aluno	GET	Pagamentos do aluno logado
/Pagamentos/{matriculaId}/confirmar	POST	Confirma pagamento (simulado)
/Progressos/aluno/{alunoId}	GET	Snapshot de progresso
/Progressos/conteudos/{id}/concluir	PUT	Marca conteúdo como concluído

NotificacoesController / CertificadosController

`NotificacoesController` (classe `[Authorize]`, qualquer role): `GET /` , `GET /nao-lidas/contagem` , `PUT /{id}/lida` , `PUT /lidas` — todos escopados ao usuário do token.

`CertificadosController` (sem `[Authorize]` de classe): `POST /matricula/{id}/emitir` e `GET /meus` (Aluno); `GET /verificar/{codigo}` público.

8. Classes, interfaces, métodos e dependências relevantes

8.1 Hierarquia de usuários — TPT (Table Per Type)

`Usuario` é abstrata e mapeada na tabela `Usuarios` com as colunas comuns. Cada subtipo tem tabela própria, ligada 1:1 por chave estrangeira implícita do EF Core (padrão TPT): **CONFIRMADO** *Models/Usuario.cs, Aluno.cs, Professor.cs, Coordenador.cs, Admin.cs; Data/Configurations/UsuariosConfiguration.cs:24-80*

Classe	Tabela	Colunas próprias
`Aluno : Usuario`	`Alunos`	`Matricula` (string), navegação `List<Matricula> Matriculas`
`Professor : Usuario`	`Professores`	`CodigoRegistro` (único), `Especialidade`
`Coordenador : Usuario`	`Coordenadores`	`CodigoRegistro` (único)
`Admin : Usuario`	`Admins`	nenhuma

O método `Usuario.ConfigurarAcesso(tipoUsuario, senhaHash, ativo)` (`Usuario.cs:96-111`) é o único ponto onde `TipoUsuario` é definido, usado uniformemente pelos 4 fluxos de criação de conta — exemplo de encapsulamento real (setter da propriedade é privado).

8.2 Padrão de repositório genérico

```
public class GenericRepository<T> : IGenericRepository<T> where T : class
{
    protected readonly PlataformaContext Context;
    protected readonly DbSet<T> DbSet;

    public async Task<T?> ObterPorIdAsync(int id) => await DbSet.FindAsync(id);
    public async Task<List<T>> ObterTodosAsync() => await DbSet.ToListAsync();
    public async Task<T> AdicionarAsync(T entidade) { await DbSet.AddAsync(entidade); return entidade; }
    public void Atualizar(T entidade) => DbSet.Update(entidade);
    public void Deletar(T entidade) => DbSet.Remove(entidade);
    public async Task<bool> SalvarAlteracoesAsync() => (await Context.SaveChangesAsync()) > 0;
}
```

Repositories/GenericRepository.cs (código integral, 34 linhas)

8.3 Serviços centrais e seus métodos mais relevantes

Classe	Dependências injetadas	Métodos-chave
`AuthService : IAuthService`	PlataformaContext, configuração JWT	`LoginAsync` , `GerarAccessToken` , `GerarRefreshTokenAsync` , `BootstrapAdminAsync`
`MatriculaService : IMatriculaService`	IMatriculaRepository, PlataformaContext, ICursoService, INotificacaoService	`MatricularComAprovacaoAutomaticaAsync` , `AprovarNoCursoAsync` , `SalvarComProtecaoDeConcorrenciaAsync`
`AvaliacaoService : IAvaliacaoService`	PlataformaContext, IProgressoAlunoService, INotificacaoService	`ListarQuestoesPorAlunoAsync` , `EnviarRespostasAlunoAsync` , `ObterRevisaoTentativaAsync`
`CertificadoService : ICertificadoService`	PlataformaContext, IAcessoAcademicoService	`EmitirCertificadoAsync`
`CursoService : ICursoService`	PlataformaContext, IGenericRepository<Curso>, IGenericRepository<Coordenador>, INotificacaoService	`GarantirCursoAbertoAsync` , `ListarProfessoresVinculadosAsync` , `ConcluirCursoComoProfessorAsync` , `EncerrarCursoAsync`

9. Persistência, entidades, relacionamentos e migrations

9.1 Entidades principais e relacionamentos

Entidade	Relacionamentos	Observação
`Curso`	1:N Módulos/Turmas/ProgressosCurso; N:1 opcional Coordenador; N:1 Admin (criador)	`EhGratuito` é `[NotMapped]`, computad Campo `ModeloCurso` (enum).
`ProfessorCurso`	N:1 Professor, N:1 Curso	Junção N:N real, índice único `(ProfessorId CursoId)`. Vínculo de "quem leciona o quê" Rodada 1 da Fase B — substitui `Turma.Prof` como mecanismo real.
`Turma`	N:1 Curso, N:1 Professor; 1:N Matrículas/Conteúdos/Avaliações	Ainda existe fisicamente e é referenciada (`ConteudoDidatico.TurmaId` / `Avaliac` continuam NOT NULL), mas **não é mais o mec matrícula** — ver 9.3 e seção 11.3.
`Matricula`	N:1 Aluno (Cascade), N:1 Curso (Restrict), N:1 opcional Turma (Restrict); 1:N Tentativas/Lançamentos/Progressos/Pagamentos	`TurmaId` é `int?` com setter privado — só `VincularTurma`, método que não é mais c nenhum fluxo vigente de aprovação.
`ConteudoDidatico` / `Avaliacao`	N:1 Professor (autor), N:1 Turma, N:1 Módulo, N:1 **Curso** (direto)	`CursoId` próprio adicionado em 22/10/202 existia via Turma/Módulo.
`QuestaoBanco` → `QuestaoPublicada`	1:N Alternativas/Afirmativas/Anexos em cada lado	`QuestaoPublicada` é snapshot imutável d `QuestaoBanco` no momento da publicação banco depois não altera avaliação já publicada.
`TentativaAvaliacao`	N:1 Avaliacao (Cascade), N:1 Matrícula (Restrict); 1:N RespostaAluno (Cascade)	Índice único `(MatriculaId, AvaliacaoI NumeroTentativa)` — é essa constraint qu tentativa concorrente duplicada falhar no banc regra de negócio.
`ProgressoConteudoAluno` / `ProgressoModuloAluno` / `ProgressoCursoAluno`	Cada um N:1 Matrícula + N:1 entidade correspondente	3 níveis de progresso persistidos separadamen com índice único por par.
`Pagamento`	N:1 Matrícula (Cascade)	Índice único em `MatriculaId` — no máxim por matrícula no modelo atual.

9.2 Enums

Enum	Valores
`TipoConteudoDidatico`	Texto, Pdf, Video, Link, Imagem
`StatusPublicacao`	Rascunho, Publicado, Arquivado
`TipoQuestao`	MultiplaEscolha, VerdadeiroFalso, Dissertativa, AfirmativasCombinadas
`TipoAvaliacao`	**Quiz** (valor padrão), Prova, Exercicio
`StatusTentativaAvaliacao`	EmAndamento, Enviada, Corrigida, Expirada
`StatusProgressoAprendizagem`	NaoIniciado, EmAndamento, Concluido
`OrigemCorrecaoNota`	Automatica, Manual, Mista
`StatusPagamento`	Pendente, Pago, Cancelado
`ModeloCurso`	Gratuito, PagoPacote, Acompanhado — define a regra de liberação de certificado (ver seção 11.7)
`StatusMatricula`	Pendente, Aprovada, Rejeitada, Cancelada
`TipoNotificacao`	MatriculaAprovada, MatriculaRejeitada, AvaliacaoCorrigida, ConteudoPublicado, PagamentoConfirmado, CursoConcluidoPeloProfessor

9.3 Linha do tempo de migrations (36 ao todo)

Fase fundacional (mar–mai/2026): `CriacaoDoBanco` → `AdicionarTipoUsuario` → `ImplementarHerancaUsuarios` (estabelece TPT) → `TornarCoordenadorOpcional` → `AdicionarColunaEspecialidade` → `VincularProfessorATurma` → `CriarTabelaMatriculas` → `AddCursoIdAndMakeTurmaNullableInMatricula` (1ª vez que Matrícula ganha CursoId) → `AjustesFinaisCursos` → `ConfigureDecimalPrecision` → `AddPedagogicalCore` (cria o núcleo pedagógico: ConteudoDidatico/Avaliacao/questões/progresso) → `AddModuloManagementIndex` → 5 migrations de código de registro (Acadêmico/Matrícula/Professor/Turma/Coordenador) → `NormalizarFeedbackAcademico` → `NormalizarPerfisUsuario`.

Hardening (ago/2026): `AdicionarIndiceUnicoEmailCpfUsuarios` → `CorrigirCascadeDeleteCursoTurmaModulo` → `RemoverTurmaAtualDoAluno` → `RemoverCursoResponsavelDoCoordenador` → `AdicionarImagemUrlCurso` → `AdicionarCertificadoEmitidoEmMatricula` → `AdicionarIndiceUnicoMatriculaAprovadaPorTurma` → `AddRefreshTokens` → `AdicionarTokenRecuperacaoSenha` → `AdicionarNotificacoes` → `RenomeiaRefreshTokenParaHash`.

Set–out/2026: `AdicionarPagamento` → `AdicionarConteudoDidaticoIdEmAvaliacao` → `TornarModuloOpcionalEmAvaliacaoELancamentoNota` → `RemoverMarcoProgressoAluno` (removeu tabela morta, nunca escrita por nenhum código) → `IndicesEEndurecimentoFksTurma` → `PadronizaCascadeProgressoParaRestrict` → `AdicionarAfirmativasQuestaoEBibliografia`.

Fase B — remoção de Turma do fluxo de matrícula (out/2026, as 3 mais recentes):

1. `AdicionarProfessorCursoENovoIndiceMatricula` (21/10) — cria `ProfessoresCursos` (N:N) com backfill a partir dos pares existentes em Turmas, + índice único por `(AlunoId, CursoId)` substituindo a proteção antiga por `TurmaId`.
2. `AdicionarCursoIdEmAvaliacoesEConteudos` (22/10) — Avaliação/Conteúdo Didático ganham `CursoId` próprio.

3. `AdicionarModeloCursoEEncerramento` (22/10) — `Curso.ModeloCurso`, `ConcluidoPeloProfessorId/Em`, `EncerradoPorId/Em`.

> **Turma/TurmaId continuam existindo fisicamente no banco** em todas as tabelas que os referenciam — nenhuma migration de remoção foi aplicada. A remoção destrutiva (`DROP COLUMN` / `DROP TABLE Turmas`) está deliberadamente pendente de um período de observação em produção antes de ser executada.

10. Autenticação, autorização, segurança e configuração

10.1 Autenticação JWT

HS256 (`SymmetricSecurityKey`), validando issuer/audience/lifetime/signing key, `ClockSkew = TimeSpan.Zero` (sem tolerância de expiração). Claims emitidas: `sub` / `NameIdentifier` , `email` , `Name` , `Role` , `usuarioId` . **CONFIRMADO** *Program.cs:146-200; AuthService.cs:210-239*

Caso especial: requisições para `/uploads/*` (imagens/PDFs em tags `<img>` / `<a>` , que não enviam header `Authorization`) aceitam o token via query string `?access_token=` . *Program.cs:189-198*

10.2 Refresh token

64 bytes aleatórios gerados por token; apenas o **hash SHA-256** é persistido em `RefreshTokens` — o valor bruto é devolvido ao cliente uma única vez e nunca mais recuperável do banco. Rotação simples a cada uso (token antigo revogado, par novo emitido).

10.3 Senhas

BCrypt (hash adaptativo com salt, via `BCrypt.Net-Next`) — nunca texto puro, nunca incluída em nenhum DTO de resposta (`SenhaHash` tem `[JsonIgnore]`).

10.4 Autorização por papel

Baseada em `[Authorize(Roles = "...")]` + checagens programáticas adicionais (`PodeAcessarAluno` , `ICursoAutorizacaoService.PodeGerenciarCursoAsync` para granularidade por curso específico do Coordenador). Resumo por papel está na seção 7.

10.5 Hardening de produção (fail-fast)

- Fora de `Development` , o boot falha imediatamente se `Jwt:Key` não tiver pelo menos 32 caracteres reais (não o placeholder versionado) — evita subir "funcionando" com chave insegura.
- Fora de `Development` , o boot falha se `Cors:AllowedOrigins` estiver vazio — evita bloqueio silencioso de CORS descoberto só em uso real.
- `ForwardedHeadersOptions` configurado para reconhecer `X-Forwarded-For` / `X-Forwarded-Proto` vindos do proxy (App Service/Front Door), com `KnownNetworks` / `KnownProxies` vazios de propósito (confiança implícita na borda do Azure, padrão documentado pela Microsoft).
- `UseHsts()` fora de Development.

10.6 Controle de acesso a arquivo

`/uploads/cursos/*` (capas de curso) é público — material de marketing, precisa aparecer na Home sem sessão. `/uploads/conteudos/*` e `/uploads/questoes/*` exigem autenticação e, para Aluno especificamente, uma checagem adicional de posse de pagamento (`IUploadAutorizacaoService`) — staff (Admin/Coordenador/Professor) só precisa estar autenticado.

10.7 Configuração

Hierarquia padrão do ASP.NET Core: `appsettings.json` → `appsettings.{Environment}.json` → User Secrets (Development) → variáveis de ambiente (convenção `Jwt__Key` , dois underscores substituem dois-pontos). `appsettings.example.json` é um modelo de referência com todas as chaves documentadas, não carregado pela aplicação.

Seção	Chaves
`ConnectionStrings`	`DefaultConnection` (fallback de desenvolvimento: LocalDB)
`Jwt`	`Key` , `Issuer` , `Audience` , `ExpireMinutes` , `RefreshTokenExpireDays`
`Cors`	`AllowedOrigins` (array)
`AzureStorage`	`ConnectionString` , `ContainerName` — vazio = usa disco local
`IA` / `Gemini`	`Provedor` , `ApiKey` , `Modelo` — chave vazia desativa a feature sem derrubar o boot
`Smtp`	`Host` , `Port` , `Usuario` , `Senha` , `Remetente` , `UsarSsl`
`RunMigrationsOnStartup`	bool — só true em Development ou docker-compose; produção aplica migration manualmente

11. Exemplos de fluxos reais extraídos do código

11.1 Login

1. `POST /Auth/login` (`[AllowAnonymous]` , rate limit `auth`) recebe `LoginDto` .
2. `AuthService.LoginAsync` busca `Usuario` por e-mail normalizado na tabela base `Usuarios` — funciona para qualquer um dos 4 tipos porque é query na raiz TPT.
3. Verifica senha com `BCrypt.Verify` ; usuário inativo → `UnauthorizedAccessException` .
4. `MontarRespostaAsync` gera access token (claims + expiração configurável) e refresh token (hash persistido).
5. Retorna `AuthResponseDto` 200, ou 401 se inválido.

AuthController.cs:20-38; AuthService.cs:27-45,191-253

11.2 Cadastro de aluno (auto-serviço)

1. `POST /Alunos/cadastro-completo` (público) → `AlunoService.CadastrarAlunoCompletoAsync` .
2. Valida e-mail/CPF únicos via `UsuarioValidacao.NormalizarEGarantirDisponivelAsync` .
3. Busca preço do curso escolhido; abre transação.
4. Cria `Aluno` com `ConfigurarAcesso("Aluno", BCrypt.HashPassword( ... ))` .
5. Chama `IMatriculaService.MatricularComAprovacaoAutomaticaAsync` (ver 11.3) e comita a transação.

AlunosController.cs:48-58; AlunoService.cs:91-126

11.3 Matrícula — aprovação automática, sem Turma (regra atual)

1. `MatriculaService.MatricularComAprovacaoAutomaticaAsync` é o ponto de entrada único, chamado tanto pelo cadastro quanto pela solicitação avulsa do Catálogo.
2. `CriarMatriculaPendenteAsync` chama `ICursoService.GarantirCursoAbertoAsync` (bloqueia se curso Acompanhado estiver encerrado) e cria a `Matricula` com `Status=Pendente` , **sem Turmald**.
3. `AprovarNoCursoAsync` : consolida duplicata se já existir matrícula Aprovada do mesmo par aluno/curso; senão chama `matricula.Aprovar()` (só seta `Status=Aprovada` — **não chama** `VincularTurma` em nenhum momento) e `CriarPagamentoPendenteSeNecessarioAsync` (cria `Pagamento` pendente só se o curso for pago).
4. `SalvarComProtecaoDeConcorrenciaAsync` traduz violação do índice único (duas aprovações simultâneas) em erro de negócio legível.

MatriculaService.cs:49-90,171-182,350-399

> Confirmação direta no código: `Matricula.VincularTurma` / `AprovarComTurma` existem na entidade, mas **não são chamados por nenhum método de aprovação atualmente em uso**. Toda matrícula aprovada pelo fluxo vigente fica com `TurmaId = null` permanentemente. O controle de acesso a conteúdo/avaliação usa `Matricula.CursoId` , não `TurmaId` .

11.4 Início e conclusão de Quiz

1. `GET /Avaliacoes/{id}/aluno/questoes` → `ListarQuestoesPorAlunoAsync` . **Não verifica limite de tentativas** — só confirma que a avaliação está publicada e acessível ao aluno.
2. `POST /Avaliacoes/{id}/aluno/respostas` → `EnviarRespostasAlunoAsync` : valida janela de disponibilidade e que todas as questões foram respondidas; **checa limite de tentativas aqui, e só aqui** (`tentativasRealizadas ≥ TentativasPermitidas` → `InvalidOperationException("O limite de tentativas para esta avaliacao ja foi atingido.")`); cria `TentativaAvaliacao` , corrige automaticamente as questões objetivas.
3. Para `TipoAvaliacao.Quiz` : chama `ProgressoAlunoService.RecalcularProgressoQuizAsync` — conta para o progresso do módulo/curso, **não gera nota nem notificação**.

AvaliacoesController.cs:130-142,236-253; AvaliacaoService.cs:565-692

11.5 Consulta da revisão de Quiz já concluído

Endpoint dedicado e **separado** do envio: `GET /Avaliacoes/{id}/aluno/tentativas/{tentativaId}/revisao` → `ObterRevisaoTentativaAsync` :

1. Carrega a tentativa com Avaliação, Matrícula, Respostas e questões/alternativas relacionadas.
2. Valida posse (`tentativa.Matricula.AlunoId == alunoId`).
3. Valida que a tentativa já foi corrigida (`StatusTentativa == Corrigida`).

AvaliacoesController.cs:255-272; AvaliacaoService.cs:694-719

> **Achado relevante para diagnóstico de bug (não corrigido neste documento, fora de escopo):** este método **não tem nenhuma checagem de limite de tentativas restantes** — é estrutural e textualmente ausente, diferente de `EnviarRespostasAlunoAsync` . Se um botão "Revisão Detalhada" no frontend estiver exibindo a mensagem "Você já usou todas as tentativas desta avaliação", essa mensagem só pode se originar do fluxo de envio (11.4) ou de uma camada de frontend não coberta por esta auditoria — o backend, isoladamente, já separa corretamente os dois fluxos.

11.6 Avaliação formal — diferenças reais vs. Quiz

Mesma trilha de endpoints/métodos. As únicas diferenças comprovadas estão dentro de `EnviarRespostasAlunoAsync` , ramificadas por `TipoAvaliacao` :

	Quiz	Prova / Exercício
Efeito ao enviar	`RecalcularProgressoQuizAsync`	`RecalcularNotaAvaliacaoAsync` (se não houver dissertativa pendente)
Notificação	nenhuma	`TipoNotificacao.AvaliacaoCorrigida`
Conta pra nota/certificado	não	sim

Estrutura de tentativa/resposta	idêntica — mesmas entidades, mesmo índice único, mesma correção objetiva

11.7 Emissão de certificado

`CertificadoService.EmitirCertificadoAsync` — pré-condições checadas em ordem (todas lançam exception se falharem):

1. Matrícula existe e pertence ao aluno.
2. `Status == Aprovada`.
3. Pagamento confirmado, se curso pago (`IAcessoAcademicoService`).
4. `ProgressoCursoAluno.PercentualConclusao ≥ 100` (métrica que já soma conteúdo + Quiz).
5. Checagem extra só para Prova/Exercício: nenhuma avaliação publicada com `TipoAvaliacao ≠ Quiz` pode estar sem `LancamentoNotaAluno` — **Quiz é explicitamente excluído** dessa validação.
6. Se `ModeloCurso == Acompanhado`, exige `EncerradoEm.HasValue` — Gratuito/PagoPacote não têm esse gate.
7. Marca `CertificadoEmitidoEm` (operação idempotente).

CertificadoService.cs:19-86

12. Limitações técnicas e funcionalidades parcialmente implementadas

PARCIAL **Repository pattern aplicado de forma inconsistente.** Só 4 entidades (de 29) têm repositório dedicado; as demais usam `PlataformaContext` direto nos Services. Decisão já documentada como deliberada em auditorias anteriores do projeto, não um descuido — mas é uma divergência real em relação a um "Repository Pattern" aplicado universalmente.

PARCIAL **Transição da Turma como unidade de matrícula ainda não finalizada.** As tabelas/colunas de Turma continuam existindo fisicamente no banco (nenhuma migration de remoção aplicada), mas deixaram de ser o mecanismo real de matrícula/acesso desde 21–22/10/2026. É um estado de transição arquitetural deliberado, não um erro — a remoção destrutiva está pendente de um período de observação em produção.

PENDENTE **Distribuição do instalador Desktop.** O instalador (`.exe` / `.dmg` / `AppImage`) precisa ser gerado manualmente (`desktop/npm run build`) e não há, no repositório, nenhum pipeline automatizado de publicação/hospedagem (como existe para o Mobile via EAS Build). Não encontrado nenhum fluxo de release automatizado.

PENDENTE **Bug relatado no botão "Revisão Detalhada" do Quiz (frontend).** O backend, auditado nesta sessão, já separa corretamente os fluxos de iniciar tentativa e consultar revisão (seção 11.5). A causa raiz de um eventual comportamento incorreto no frontend não foi investigada nesta auditoria — está fora do escopo deste manual, que cobre especificamente C#/Desktop.

PENDENTE `DevelopmentDataSeeder.cs` não foi auditado por completo quanto à consistência com o fluxo de matrícula independente de Turma (ver 12, item anterior) — não confirmado se os dados de desenvolvimento seguem a mesma regra que a produção.

13. Roteiro de apresentação acadêmica (10–15 minutos)

Tempo	Bloco	Pontos a cobrir
0–1 min	Contexto	O que é o PlataformaEnsino/CodeRyse Academy, por que existem 3 clientes (Web/Mobile/Desktop) sobre o mesmo backend C#.
1–3 min	Arquitetura em camadas	Controllers → Services → Repositories/DbContext → Models, com interfaces e DI (seção 4). Mostrar `Program.cs` ao vivo (os 30 registros de serviço).
3–5 min	Modelo de domínio — POO real	Hierarquia `Usuario` (TPT, abstração/herança/encapsulamento — seção 8.1 e 9.1). É o exemplo mais forte de POO do projeto.
5–7 min	Segurança	JWT + refresh token + BCrypt + rate limiting + fail-fast de configuração (seção 10).
7–10 min	Demonstração ao vivo do Desktop	Abrir o app Desktop empacotado, logar, navegar — mostrando que é a mesma SPA da Web rodando numa janela nativa Electron (seção 5–6).
10–13 min	Um fluxo de negócio completo	Escolher matrícula OU quiz (seção 11) e narrar Controller → Service → banco, com o código aberto.
13–15 min	Encerramento + perguntas	Resumo de limitações conhecidas (seção 12) — mostra maturidade técnica reconhecer o que está parcial.

14. Perguntas prováveis da banca e respostas fundamentadas

O aplicativo Desktop é uma aplicação C# nativa (WPF/WinForms/MAUI)?

Não. É um wrapper **Electron** (JavaScript/Node, empacotado via `electron-builder`) que carrega, numa janela nativa, a mesma SPA React que a Web usa. A única aplicação C# do projeto é o backend (API). O Desktop não contém nenhuma linha de C#.

Por que não usar WPF, WinForms ou .NET MAUI para o Desktop?

Decisão de reaproveitamento: o frontend React já estava pronto e validado contra o design do projeto; reescrevê-lo numa stack nativa .NET duplicaria toda a interface sem agregar valor arquitetural — o foco da solução está no backend C# único servindo múltiplos clientes, não em qual tecnologia de UI cada cliente usa.

Como funciona a herança de usuários no banco de dados?

Table Per Type (TPT): `Usuario` é abstrata, mapeada na tabela `Usuarios` com as colunas comuns; cada subtipo (`Aluno`, `Professor`, `Coordenador`, `Admin`) tem sua própria tabela, ligada 1:1 pela chave primária — e só guarda as colunas específicas daquele papel.

Por que o Repository Pattern não é usado para todas as entidades?

Decisão deliberada: repositório dedicado só é criado quando há consulta reutilizável ou complexa o bastante pra justificar a camada extra (4 entidades hoje). Para o resto, o Service acessa o `DbContext` diretamente — reduz indireção sem abrir mão da separação Controller/Service.

Como a senha do usuário é protegida?

Hash com BCrypt (algoritmo adaptativo, com salt automático) — nunca texto puro, nunca retornada em nenhuma resposta da API (a propriedade tem `[JsonIgnore]`).

O que acontece quando o token JWT expira?

O cliente chama `POST /Auth/refresh` com o refresh token (válido por mais tempo, configurável), que valida o hash armazenado, revoga o token antigo e emite um par novo — sem exigir novo login.

Por que existe uma tabela Turma se a matrícula não depende mais dela?

Decisão de migração incremental e segura: Turma era, até outubro/2026, o mecanismo de matrícula e controle de acesso. A migração para um modelo direto Aluno→Curso foi feita em 3 etapas aditivas (sem remover nada do schema antigo), para permitir rollback e validação em produção antes de uma remoção destrutiva futura.

Quiz e Avaliação formal são entidades diferentes?

Não — são a mesma entidade `Avaliacao`, diferenciadas pelo campo `TipoAvaliacao` (enum). A diferença de comportamento está isolada dentro do método que processa o envio de respostas.

15. Referências — arquivos e métodos utilizados

`Sistema Academico Integrado.csproj` · `Program.cs` · `appsettings.example.json` · `desktop/main.js` · `desktop/package.json` · `desktop/README.md` · `Common/ApiExceptionMiddleware.cs` · `Common/ClaimsPrincipalExtensions.cs` · `Common/RequestLoggingMiddleware.cs` · `Repositories/GenericRepository.cs` · `Data/PlataformaContext.cs` · `Data/Configurations/UsuariosConfiguration.cs` · `Data/Configurations/AcademicoConfiguration.cs` · `Data/DevelopmentDataSeeder.cs` · todos os 15 arquivos em `Controllers/` · `Services/AuthService.cs` · `Services/MatriculaService.cs` · `Services/AvaliacaoService.cs` · `Services/CertificadoService.cs` · `Services/CursoService.cs` · `Models/Usuario.cs`, `Aluno.cs`, `Professor.cs`, `Coordenador.cs`, `Admin.cs`, `Curso.cs`, `ProfessorCurso.cs`, `Turma.cs`, `Matricula.cs`, `ConteudoDidatico.cs`, `Avaliacao.cs`, `QuestaoBanco.cs`, `QuestaoPublicada.cs`, `TentativaAvaliacao.cs`, `RespostaAluno.cs`, `PedagogicalEnums.cs`, `StatusMatricula.cs`, `Notificacao.cs`, `Pagamento.cs` · todas as 36 migrations em `Migrations/`.

Glossário

TPT (Table Per Type)
Estratégia de mapeamento objeto-relacional em que cada classe de uma hierarquia de herança tem sua própria tabela, ligada por chave primária compartilhada.

DI (Dependency Injection)
Injeção de dependência — padrão em que uma classe recebe suas dependências (via construtor) em vez de instanciá-las, geralmente resolvidas por um container central (aqui, o do ASP.NET Core).

DTO (Data Transfer Object)
Objeto usado para transportar dados entre camadas/API, sem expor diretamente a estrutura interna da entidade de domínio.

JWT (JSON Web Token)
Token assinado digitalmente usado para autenticação stateless — o servidor não precisa guardar sessão, só validar a assinatura.

Snapshot (QuestaoPublicada)
Cópia imutável de uma questão do banco, feita no momento da publicação de uma avaliação, para que edições futuras no banco de questões não alterem provas já aplicadas.

Rate limiting
Mecanismo que restringe o número de requisições permitidas num intervalo de tempo, por IP ou por usuário.

Fail-fast
Estratégia de interromper a inicialização da aplicação imediatamente ao detectar uma configuração inválida, em vez de seguir rodando num estado inseguro.